

Theory Assignment 4

Aayush Kumar - CO21BTECH11002

April 2025

1 Q1 (5.2)

We start with the version orders $x_0 \ll x_1 \ll x_3$ and $y_0 \ll y_1 \ll y_2$. Now we can build the $MVSG(m, \ll)$ graph:

- We add the edge $t_0 \rightarrow t_1$ because of the operation $r_1(x_0)$.
- Then we look at $r_1(x_0)$ and $w_3(x_3)$. Since $x_3 \gg x_0$, we add the edge $t_1 \rightarrow t_3$.
- We add the edge $t_1 \rightarrow t_2$ because $r_2(x_1) \in op(m)$.
- Looking at $r_2(x_1)$ and $w_0(x_0)$, we add the edge $t_0 \rightarrow t_1$, since $x_0 \ll x_1$.
- We now consider $r_2(x_1)$ and $w_3(x_3)$, and add the edge $t_2 \rightarrow t_3$, because $x_3 \gg x_1$.

The graph obtained is shown in Figure 1. As the graph is acyclic, $x_0 \ll x_1 \ll x_3$ and $y_0 \ll y_1 \ll y_2$ are valid version orders.

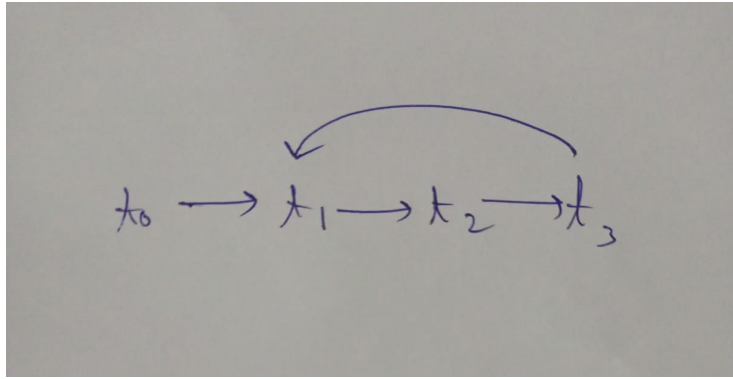


Figure 1: $MVSG(m, \ll)$ graph

Finally, for t_∞ , we choose the following version function:

- $h(r_\infty(x)) = w_3(x)$
- $h(r_\infty(y)) = w_2(y)$

Hence, we add the edge $t_3 \rightarrow t_\infty$, $t_2 \rightarrow t_\infty$, and $t_0 \rightarrow t_2$.

The final graph is shown in Figure 2. The version function H is a valid version function for the $MVSG(m, \ll)$ graph since the graph remains acyclic.

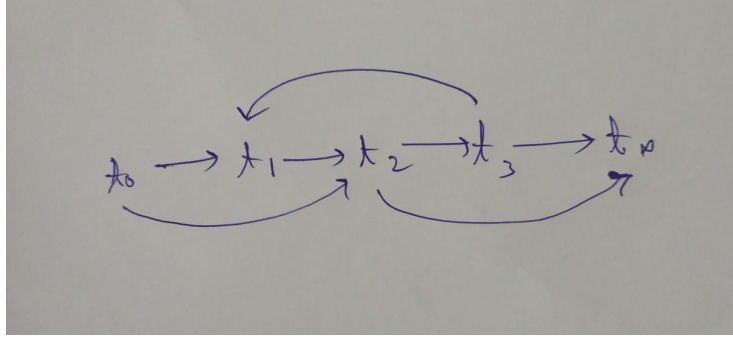


Figure 2: Final $MVSG(m, \ll)$ graph

2 Q2 (5.6)

The input schedule is:

$$w_1(x) \ c_1 \ r_2(x) \ r_3(x) \ c_2 \ r_4(x) \ w_3(x) \ c_4 \ c_3$$

We analyze this schedule using the MVTO (Multiversion Timestamp Ordering) protocol:

- Transaction t_1 performs a write $w_1(x)$, and then commits with c_1 . This creates version x_1 .
- Transaction t_2 performs a read $r_2(x)$. It reads version x_1 , which is valid since t_1 has committed before t_2 starts. So, t_2 proceeds and eventually commits with c_2 .
- Transaction t_3 performs a read $r_3(x)$, also reading version x_1 , which is valid, so it continues execution.
- Transaction t_4 performs a read $r_4(x)$, also reading version x_1 , which is valid.
- Transaction t_3 then tries to write $w_3(x)$. However, this would create version x_3 , which is not valid because t_4 has already read x_1 and hence, t_3 will come between t_1 and t_4 which is not allowed in MVTO. Therefore, t_3 must abort.

- Transactions t_4 and t_2 can successfully commit as they do not violate any MVTO rules.

Hence, in the output schedule, t_3 aborts, and the rest of the transactions proceed according to MVTO. Figure 3 shows the final execution.

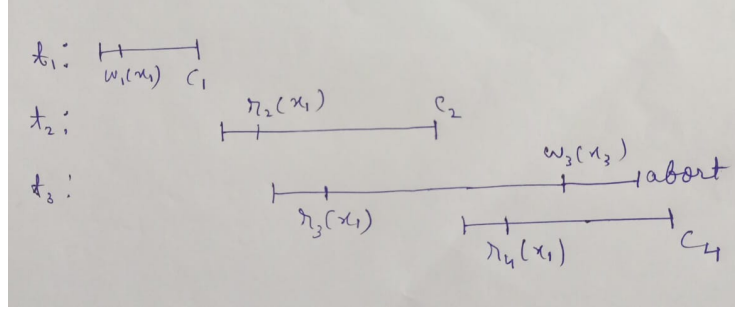


Figure 3: MVTO Execution

3 Q3 (5.9)

Consider the following history:

$$m = r_1(x_0)r_2(y_0)w_1(y_1)c_1w_2(x_2)c_2$$

The modified ROMV protocol allows above execution since the transactions do not acquire lock for reading. So both the read operations $r_1(x_0)$ and $r_2(y_0)$ will complete. Then transaction t_1 will acquire the lock on x_1 and write $w_1(x_1)$, and then commit with c_1 . After that, transaction t_2 will acquire the lock on y_2 and write $w_2(y_2)$, and then commit with c_2 . However, there is no monoversion serial schedule for which this history is serializable. Since, if we take serial history to be t_1 followed by t_2 , then the read operation $r_2(y_0)$ will not be valid since t_1 has already committed and written $w_1(y_1)$. Similarly, if we take t_2 followed by t_1 , then the read operation $r_1(x_0)$ will not be valid since t_2 has already committed and written $w_2(x_2)$. Hence, there is no monoversion serial schedule for which this history is serializable.

Thus, the modified ROMV protocol is not correct.